

# **Aligning Behavioural Heuristics with Agentic RAG for Culturally Adapted Consumer Simulation**

Technical Submission Paper

## Table of Contents

|                                               |          |
|-----------------------------------------------|----------|
| <b>Abstract</b>                               | <b>3</b> |
| 1. Problem Statement                          | 4        |
| 2. Design Goals                               | 4        |
| 3. Architecture and Implementation            | 4        |
| 4. Data Preparation and Persona Construction  | 6        |
| 5. Retrieval Grounding                        | 6        |
| 6. Generation Strategy                        | 7        |
| 7. Evaluation Strategy                        | 8        |
| 8. Experimental Setup and Development History | 9        |
| 9. Representative Successful Run Outcomes     | 9        |
| 10. Limitations                               | 10       |
| 11. Reproducibility and Deployment            | 11       |
| 12. conclusion                                | 11       |
| 13. Reference                                 |          |

## Abstract

The Behavioural Intelligence Agent addresses a specific systems problem: how to simulate consumer reasoning and review writing in a way that stays grounded in behavioural signals rather than drifting into generic model output. The repository combines a core Python library, a FastAPI backend, and a multipage Streamlit front end. Its central design choice is to separate the understanding of a shopper from the generation of text. A deterministic persona builder turns free-text inputs, review history, and sentiment cues into a structured UserPersona. Retrieval then uses behavioural metadata stored in Chroma rather than a free-form semantic query. Generation is handled by a LangGraph ReAct review agent, while a separate Hugging Face based path supports recommendation chat. The result is a layered, explainable architecture that supports localized consumer simulation, experimentation, evaluation, and deployment in a single codebase.

## 1. Problem Statement

Generic large language models are effective at fluent text generation, but they are weak at preserving the small behavioural details that make a consumer profile believable. When the task is to simulate a particular shopper, the model must retain stable traits such as rating bias, verbosity, tone, concern themes, preferred features, and regional style markers. A purely prompt-based approach often collapses these traits into broad stereotypes or inconsistent prose. The repository therefore targets a narrower but more useful problem: produce reviews and recommendations that are grounded in a structured behavioural profile and in analogous historical review data.

The practical motivation is twofold. First, the system is intended to generate review-like outputs that are coherent enough for downstream product or consumer research workflows. Second, it must support a recommendation experience that remembers prior context without requiring the user to restate the same persona details on every turn. This pushes the implementation toward explicit state management, deterministic persona inference, and retrieval controls that can be inspected and tested.

## 2. Design Goals

The repository pursues four design goals. The first is controllability: the system should expose how a persona is inferred and how retrieved evidence is selected. The second is modularity: business logic must remain in `src/` so it can be reused from the API, the UI, notebooks, or tests. The third is grounding: review generation must draw from indexed behavioural examples rather than inventing style from scratch. The fourth is deployability: the same codebase should run locally, in Docker, and through a web app without changing the underlying algorithmic path.

## 3. Architecture and Implementation

The repository follows a library-first architecture. The FastAPI service and the Streamlit interface are thin delivery layers over the actual application logic. The core pipeline is composed of persona construction, context retention, metadata-driven retrieval, review generation, recommendation generation, and evaluation. Offline notebooks and batch scripts prepare the processed data, build the vector index, and support experiments.

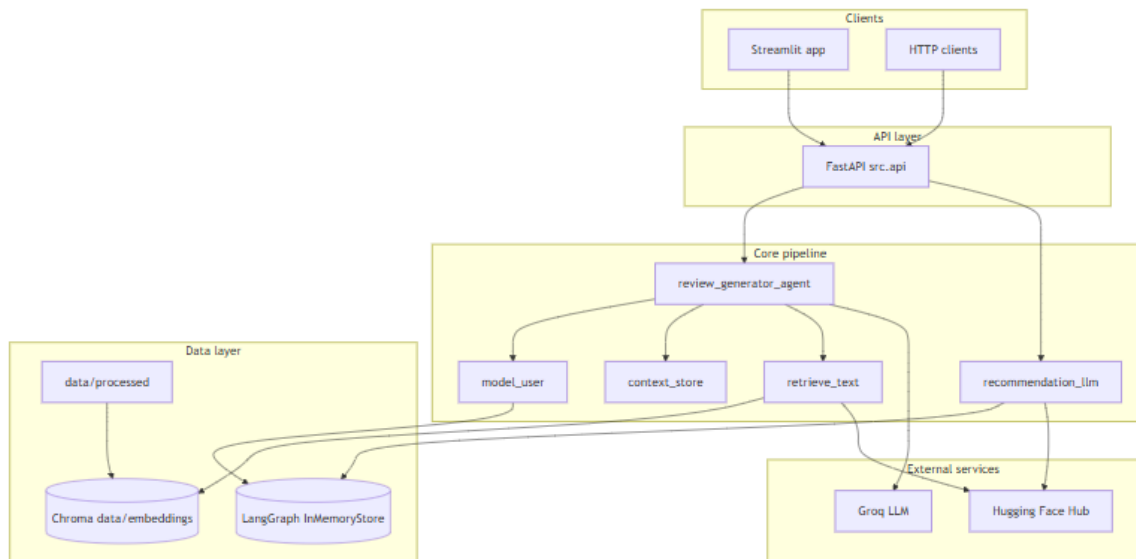


Image 1: system architecture

| Component                 | Role in the system                                                                                                  |
|---------------------------|---------------------------------------------------------------------------------------------------------------------|
| Persona builder           | Turns free-text persona prompts, review history, product context, and sentiment cues into a structured UserPersona. |
| Context store             | Persists the inferred persona in an in-memory LangGraph store for multi-turn recommendation and review flows.       |
| Retrieval layer           | Reads Chroma records with exact behavioural metadata filters instead of a semantic search string.                   |
| Review generation         | Uses a LangGraph create_react_agent loop to produce a predicted rating and grounded review draft.                   |
| Recommendation generation | Uses Hugging Face chat completions to draft recommendation responses from stored persona context.                   |
| Evaluation layer          | Runs LLM-as-judge scoring for helpfulness and plan adherence through OpenEvals.                                     |
| Streamlit UI              | Provides review generation, recommendation chat, evaluation, and persona exploration pages.                         |
| FastAPI service           | Exposes health and generation endpoints consumed by the UI and by external clients.                                 |

Table1: components and their roles

The implementation notes in the repository document the key external dependencies: Groq-backed model access for review generation, Hugging Face access for embeddings and recommendation chat, and a persistent Chroma collection for stored review histories. The result is not a monolithic agent. It is a composed system with clear seams between state handling, retrieval, generation, and presentation.

## 4. Data Preparation and Persona Construction

The data layer is built around a flattened persona library stored under `data/processed/`. The processed artifacts include a JSON representation, tabular exports, a cleaned index file for Chroma, and an encoded wide table used for classical experiments. Each record corresponds to one user and typically carries up to five historical review texts together with behavioural profile fields and Nigerian adaptation fields.

The persona builder is intentionally deterministic. Instead of calling an LLM to infer identity or style, the system applies heuristic rules to the input signals. Rating bias is estimated from explicit star references when available, otherwise it is mapped from sentiment cues. Verbosity is approximated from word count, tone is inferred from keyword scanning, and concerns or preferences are extracted from positive and negative marker phrases. This makes the resulting persona easier to explain and debug than a purely generative profile.

This choice matters because the rest of the stack depends on the persona schema being stable. The review generator, recommendation flow, and UI preview all expect the same contract. A predictable persona object also makes it possible to compare runs, perform tests, and trace why a particular retrieval filter or generated statement was chosen.

## 5. Retrieval Grounding

Retrieval is metadata-first rather than query-text-first. The user profile is translated into exact metadata predicates over Chroma fields such as rating consistency, sentiment bias, verbal style, persona type, and slang markers. That means the system is not trying to find semantic neighbors through an embedding query at generation time. Instead, it is isolating the subset of historical reviews that match the inferred behavioural signature. The historical reviews are stored as document payloads within a persistent Chroma collection, but the query string itself is discarded during retrieval. Instead, the inferred attributes of the `UserPersona` are mapped into a structured query language using standard Chroma metadata operators using exact matching (`$eq`, `$in`, `$and`).

This approach preserves discrete traits that are often diluted by generic vector search. It also narrows the evidence space for the generator, which is valuable when the goal is not open-domain retrieval but stable persona simulation. The retrieved review texts can then be summarized before being passed onward to the agent, reducing prompt size while retaining the behavioural signal.

## 6. Generation Strategy

The review path uses LangGraph's `create_react_agent` orchestration pattern. The agent is given tools for persona modelling, state persistence, and retrieval, and it can iterate through these tools before producing a final response. That structure matters because the system is not merely generating text from a single prompt. It is using an agent loop to make the generated review depend on both the inferred persona and the retrieved historical analogues.

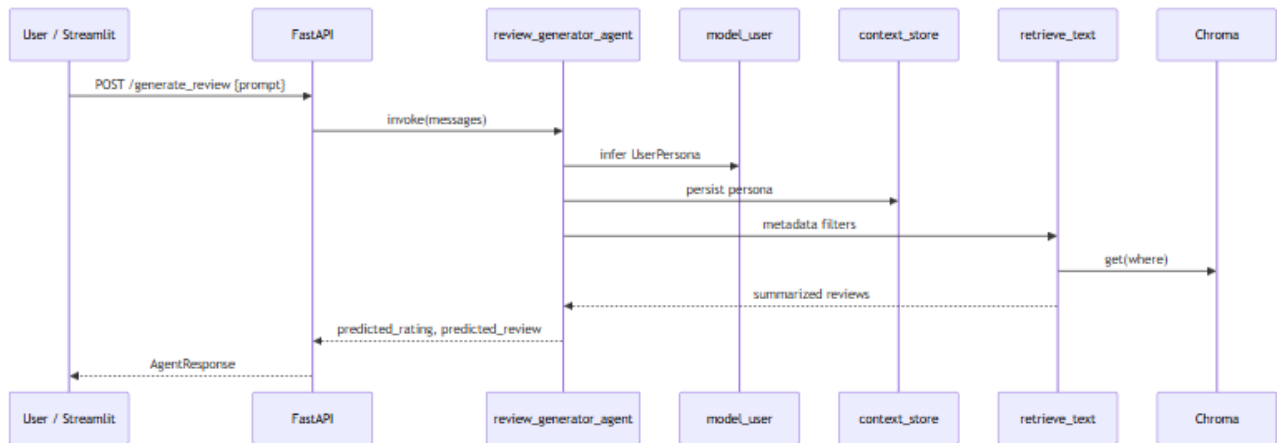


Image 2: review simulation path

The recommendation path is intentionally separate. It relies on a Hugging Face inference client and a recommendation prompt template that reuses stored persona context and session history. Separating the two generation modes prevents the review workflow from being overloaded with chat behaviour and keeps the recommendation flow optimized for conversational continuity.

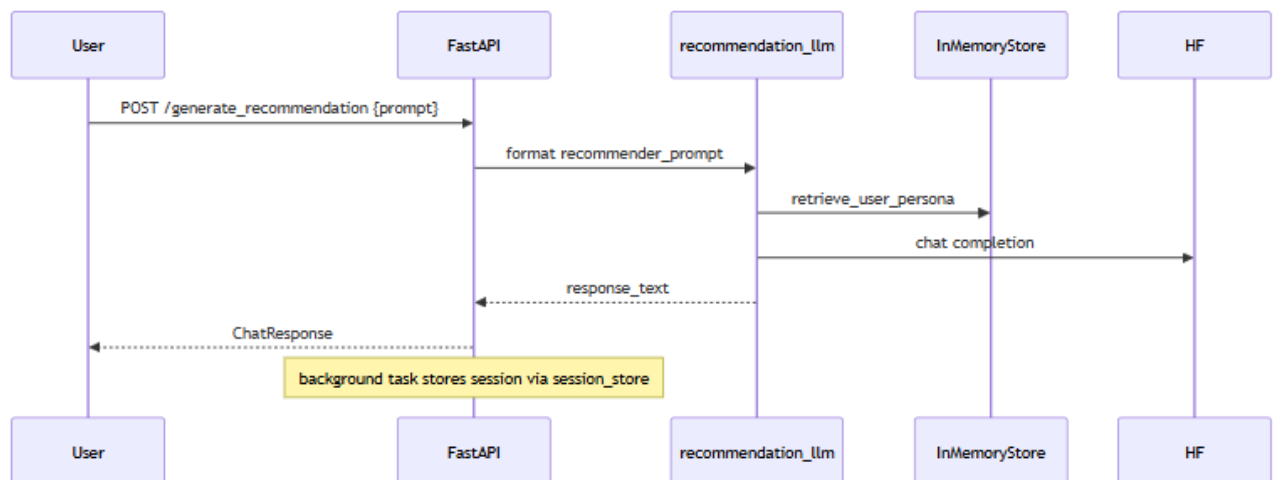


Image 3: recommendation generation path

In practice, the generation layer is where the system moves from structured representation to natural language output. The promise of the design is that the text remains stylistically plausible while still being anchored to the behavioural state established upstream.

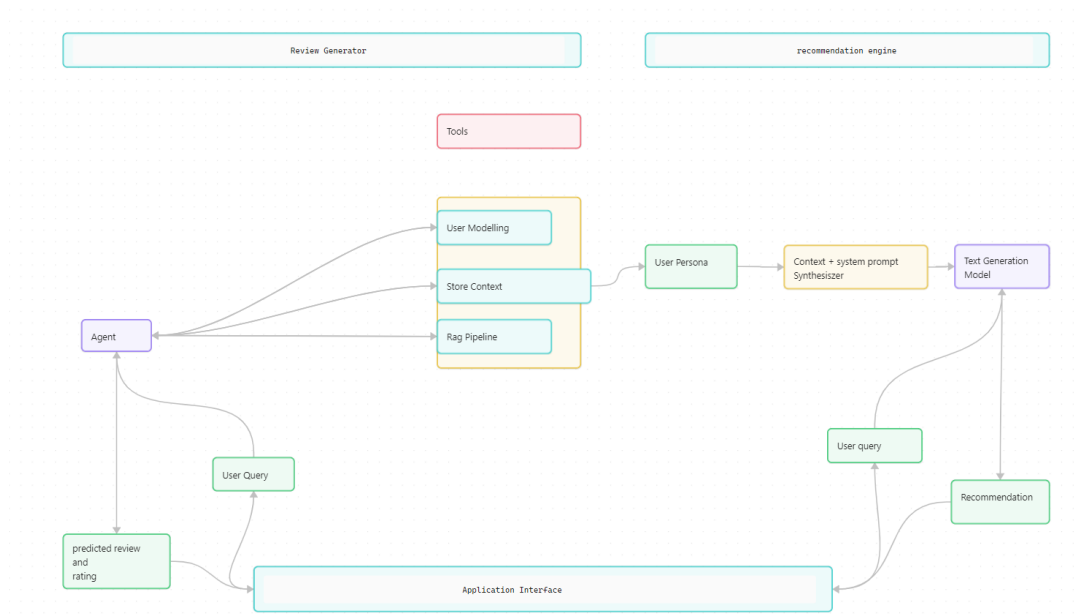


Image 4: system design

## 7. Evaluation Strategy

Evaluation is treated as a first-class module rather than a manual afterthought. The repository uses OpenEvals judges to score prompt-output pairs on helpfulness and plan adherence. This does not replace human judgement, but it provides a consistent local metric layer for comparing runs and checking whether a generated response stays on task.

The experimental notebooks also include a classical machine learning baseline. The encoded behavioural feature matrix is used in a Random Forest experiment to test whether the structured attributes contain real predictive signal. The point of the baseline is not to outperform the agent. It is to verify that the behavioural features are meaningful before they are used as scaffolding for generation and retrieval.



## 8. Experimental Setup and Development History

The repository's notebooks and scripts show a staged development process. Exploration notebooks support data inspection and cleaning, while later notebooks focus on evaluation prototypes and the Random Forest baseline. The batch scripts handle index building, encoding, and export-oriented workflows, which keeps the interactive app separate from offline data preparation.

The hypothesis space documented in the experiments notes is practical. It asks whether behavioural metadata is sufficient for retrieval, whether rule-based persona inference can steer review tone and rating, whether encoded features contain useful structure, and whether LLM judges can approximate usefulness-related judgements. That is the right framing for a project of this type because it tests each architectural claim independently.

The test suite covers the most important seams in the system: persona building, retrieval behaviour, generation paths, and the API surface. That testing strategy is aligned with the library-first design, because the core logic can be exercised without going through the web UI.

## 9. Representative Successful Run Outcomes

The public repository does not provide a single consolidated benchmark table for the live application, so a faithful paper should describe successful run patterns rather than invent exact scores. A successful review-generation run produces four visible signals: a structured persona preview, retrieved review histories that match the behavioural filters, a predicted rating, and a review draft grounded in the retrieved context.

A successful recommendation run preserves prior context across turns and returns a response that reflects the stored persona instead of re-deriving it from scratch. A successful API health check returns the documented success payload, and the Streamlit interface can route the same request through the browser, giving a human operator a direct validation path.

## 10. Limitations

The main limitation is persistence. The in-memory LangGraph store is process-local, so conversation state is not durable across restarts or horizontally scaled replicas. That is acceptable for a prototype or single-instance deployment, but it becomes a constraint in production environments that require distributed session continuity.

A second limitation is external dependency. Review generation and several auxiliary functions depend on external model services, so latency, rate limits, and service availability are partly outside local control. The system is therefore only as stable as its upstream providers.

A third limitation is the retrieval design itself. Metadata-only filtering preserves trait fidelity, but it can miss useful evidence when the behavioural schema is incomplete, noisy, or poorly mapped from the input persona. That trade-off is acceptable for strict persona simulation, but it reduces recall relative to more permissive semantic search.

Lastly, the strongest claims that can be made are architectural and operational rather than population-level empirical, as emphasis is laid on prototype execution, notebooks, and demo flows more than large-scale human evaluation.

## 11. Reproducibility and Deployment

The deployment notes are explicit. The project supports local virtual environments, editable installation, vector index building, FastAPI startup, and Streamlit startup. The Docker configuration builds from python:3.12-slim, exposes API and UI ports, and mounts data, models, and logs so the Chroma index and outputs can persist across runs.

The configuration layer reads environment variables such as `GROQ_API_KEY` and `HUGGINGFACE_API_KEY` from `.env`, which makes secrets management straightforward in development and containerized deployment. The health endpoint is designed for simple readiness checks, and the API exposes generation endpoints that can be driven either directly or through the UI.

## 12. Conclusion

This project should best be understood as a layered behavioural simulation system rather than as a generic RAG demo. Its contribution is the separation of concerns between persona inference, metadata-grounded retrieval, contextual generation, and evaluation. That separation makes the system easier to inspect, easier to test, and better suited to localized consumer simulation than a single-pass prompt workflow.

Our team's strongest design decision is the use of deterministic persona construction and metadata isolation. Together they preserve user traits that would otherwise be blurred by embedding-only retrieval. The result is a controlled architecture that can generate grounded reviews, maintain recommendation context, and support both interactive and batch experimentation from the same codebase.

## 13. Reference

Repository link:

[kinola-IQ/behavioral-intelligence-agent: The repository presents a recommendation workflow built around structured persona creation, context retention, retrieval augmentation, and prompt synthesis. The architecture is suitable for systems that need personalized outputs while keeping the generation stage controlled and explainable at a systems level.](#)

Webapp link:

<https://behavioral-intelligence-agent.streamlit.app/>

Github Container link:

<https://github.com/users/kinola-IQ/packages/container/package/behavioral-intelligence-agent-backend>

Docker image name:

kinolaiq/behavioural-intelligence-agent-backend